Data Storytelling & Interactive Dashboards

Suman 28 Feb, 2026 At 8:00 PM Notes Module-1 Recommended Resource

Associated Lectures (1)

Description Discussions

Session 5.2: Data Visualization Essentials (Matplotlib & Seaborn)

PPT File: Click Here

Colab File: Click Here

Program: Vishlesan i-Hub IIT Patna x Masai School -- AIM (AI & Machine Learning)
Session ID: 5.2 | **Week:** 5
Prerequisites: Pandas Basics, Matplotlib Introduction

Mental Map

This session focuses on mastering core visualization types and customization using Matplotlib and Seaborn.

```
┌─────────────────────────────────────────────┐
│              SESSION 5.2 FLOW              │
└─────────────────────────────────────────────┘

Topic A                  Topic B                  Topic C
┌─────────────┐          ┌─────────────┐          ┌─────────────┐
│ Core Plot   │    →     │ Seaborn     │    →     │ Customization│
│ Types       │          │ Statistical │          │ & Subplots  │
│             │          │ Plots       │          │             │
│ • Line      │          │ • Distplots │          │ • Colors    │
│ • Bar       │          │ • Boxplot   │          │ • Labels    │
│ • Scatter   │          │ • Violin    │          │ • Legends   │
│ • Histogram │          │ • Countplot │          │ • Layout    │
└─────────────┘          └─────────────┘          └─────────────┘
```

The Big Picture

Visualization is not about making charts — it is about answering questions visually.

This session focuses on:

- Choosing the correct basic plot (line, bar, scatter, histogram)
- Using Seaborn for statistical visualizations
- Customizing plots (colors, labels, legends)
- Creating multi-plot layouts using subplots

These are foundational skills required before moving to dashboards or advanced analytics.

Main Content

Topic A: Core Plot Types (Matplotlib)

Each plot answers a different analytical question.

1. Line Plot

Use When: Showing trends over time or ordered data.

Question Type:

> How does a variable change over time?

```
import matplotlib.pyplot as plt

plt.plot(df['month'], df['revenue'])
plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.show()
```

Key Points:

- X-axis usually represents time.
- Good for continuous data.
- Can add markers using `marker='o'`.

2. Bar Plot

Use When: Comparing categories.

Question Type:

> How do categories compare?

```
plt.bar(df['category'], df['sales'])
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.show()
```

Best Practices:

- Use horizontal bar chart if many categories:

```
plt.barh(df['category'], df['sales'])
```

- Sort values for better readability.

---

3. Scatter Plot

Use When: Studying relationship between two variables.

Question Type:

> Is there a relationship between X and Y?

```
plt.scatter(df['price'], df['rating'])
plt.title("Price vs Rating")
plt.xlabel("Price")
plt.ylabel("Rating")
plt.show()
```

Interpretation:

- Pattern shows correlation.
- Use transparency if many points:

```
plt.scatter(x, y, alpha=0.5)
```

---

4. Histogram

Use When: Understanding distribution of a variable.

Question Type:

> What is the distribution of values?

```
plt.hist(df['order_value'], bins=20)
plt.title("Order Value Distribution")
plt.xlabel("Order Value")
plt.ylabel("Frequency")
plt.show()
```

Key Concepts:

- `bins` controls grouping.
- Shows skewness and spread.

Topic B: Seaborn Statistical Plots

Seaborn builds on Matplotlib and provides better aesthetics and statistical plots.

```
import seaborn as sns
```

---

1. Countplot

Shows frequency of categories.

```
sns.countplot(x='category', data=df)
```

Use when:

> How many observations exist per category?

---

2. Boxplot

Shows median, quartiles, and outliers.

```
sns.boxplot(x='category', y='price', data=df)
```

Use when:

> How is distribution different across categories?

---

3. Violin Plot

Shows distribution shape + boxplot info.

```
sns.violinplot(x='category', y='price', data=df)
```

Use when:

> What is the density distribution across groups?

---

4. Scatterplot (Seaborn)

Supports grouping with hue.

```
sns.scatterplot(x='price', y='rating', hue='category', data=df)
```

Automatically adds legend.

Topic C: Customization (Colors, Labels, Legends)

Professional plots require clear formatting.

---

1. Colors

Matplotlib:

```
plt.bar(x, y, color='skyblue')
```

Seaborn palette:

```
sns.set_palette("colorblind")
```

Custom palette:

```
sns.barplot(x='category', y='sales', data=df, palette='Set2')
```

---

2. Titles and Labels

```
plt.title("Revenue by Category", fontsize=14)
plt.xlabel("Category", fontsize=12)
plt.ylabel("Revenue (INR)", fontsize=12)
```

Clear labeling improves readability.

---

3. Legends

```
plt.legend(title="Category")
```

For Seaborn:

```
sns.scatterplot(x='price', y='rating', hue='category', data=df)
plt.legend(title="Category")
```

---

4. Removing Clutter

```
sns.despine()
plt.grid(alpha=0.3)
```

Avoid excessive gridlines or 3D effects.

Topic D: Subplots (Multiple Plots in One Figure)

Used to compare multiple visualizations together.

Method 1: Using plt.subplot()

```
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.hist(df['price'])
plt.title("Price Distribution")

plt.subplot(1,2,2)
plt.hist(df['rating'])
plt.title("Rating Distribution")

plt.tight_layout()
plt.show()
```

Method 2: Using plt.subplots()

Recommended method.

```
fig, ax = plt.subplots(2, 2, figsize=(10,8))

ax[0,0].plot(df['month'], df['revenue'])
ax[0,0].set_title("Revenue Trend")

ax[0,1].bar(df['category'], df['sales'])
ax[0,1].set_title("Sales by Category")

ax[1,0].hist(df['price'])
ax[1,0].set_title("Price Distribution")

ax[1,1].scatter(df['price'], df['rating'])
ax[1,1].set_title("Price vs Rating")

plt.tight_layout()
plt.show()
```

Why use subplots?

- Compare variables side by side.
- Build dashboards in static format.
- Improve analytical clarity.

Key Frameworks

1. Chart Selection Guide

Question	Plot Type
Trend over time	Line plot
Compare categories	Bar plot
Relationship between variables	Scatter plot
Distribution	Histogram
Category frequency	Countplot
Compare distributions across groups	Boxplot

2. Clean Visualization Checklist

Before finalizing a plot:

- Is the correct chart type used?
- Are axes clearly labeled?
- Is the title meaningful?
- Are colors readable?
- Is the legend clear?
- Is clutter removed?
- Are categories sorted logically?

Common Errors Quick Reference

---

1. Using Line Plot for Categories

Incorrect:

```
plt.plot(['A','B','C'], [10,20,30])
```

Categories are not continuous — use bar chart instead.

---

2. Too Many Colors

Avoid random color usage. Stick to consistent palette.

---

3. Overlapping Labels

Rotate labels if needed:

```
plt.xticks(rotation=45)
```

---

4. Forgetting plt.tight_layout()

Without it, labels may overlap.

Key Takeaways

- Line → trend
- Bar → category comparison
- Scatter → relationship
- Histogram → distribution
- Seaborn adds statistical power and better defaults
- Customization improves clarity and professionalism
- Subplots allow multi-chart comparison

Mastering these basics ensures you can confidently build clean, insightful visualizations before moving to interactive dashboards or advanced analytics.

Vishlesan i-Hub IIT Patna × Masai School